

ELASTIC SCATTERING PLUGINS FOR THE OPENSTB SONAR SIMULATOR

C. A. Hurtado Erasso	Royal Military Academy, Université catholique de Louvain, Brussels, Louvain-la-Neuve, Belgium
Blair Bonnett	Helmut Schmidt University, Hamburg, Germany
Olga Lopera Tellez	Royal Military Academy, Brussels, Belgium
Sébastien Lambot	Université catholique de Louvain, Louvain-la-Neuve, Belgium
Xavier Neyt	Royal Military Academy, Brussels, Belgium

1 ABSTRACT

The openSTB (open sonar toolboxes) simulator is an open-source framework designed to facilitate community-driven development of sonar simulation techniques. Built with a modular plugin architecture, it enables researchers to customise simulations by combining different components for signal generation, propagation modeling, and target representation. Currently, the simulator primarily uses point-target models, which, while computationally efficient, do not capture the complex acoustic behaviour of elastic objects.

This paper presents new plugins that extend the openSTB simulator with physics-based target modeling using elastic acoustic scattering theory. A rigid sphere scattering plugin is fully integrated into the simulator pipeline, computing a frequency-dependent form function that preserve phase information and resonance phenomena, and generating the corresponding time-domain echo signals through inverse Fourier transformation. Additionally, implementations for solid elastic spheres, vacuum-filled elastic spherical shells, fluid-filled elastic spherical shells, rigid cylinders, and solid elastic cylinders are provided in the repository as validated standalone modules, establishing the foundation for their future integration into the pipeline.

Validation was performed by comparing outputs against published results from classical acoustic scattering literature, demonstrating excellent agreement across all target types. For the rigid sphere, this validation extends to the full simulator pipeline, confirming end-to-end correctness. These plugins enable more realistic simulations for applications in underwater mine detection, object classification algorithm development, and synthetic aperture sonar processing. Following the openSTB design principles, the implementation is extensible and can be combined with existing simulator components.

2 INTRODUCTION

Underwater mines and unexploded ordnance (UXO) remain a persistent threat in modern maritime environments. Multiple mine types — including drifting, moored, and seabed mines — have been deployed for several decades and continue to be laid today. The scale of historical and recent contamination is massive. Ordnance was produced in mass quantities during the World Wars and subsequently dumped at sea, predominantly in Northern Europe; for instance, it is estimated that at least 170,000 tons of chemical weapons were dumped in the Skagerrak Strait and another 50,000 tons in the Baltic Sea ¹. Furthermore, the rate at which new devices are laid or abandoned continues to drastically outpace clearance efforts, with studies indicating that naval mine clearance processes can take up to 200 times the cost and time required to initially place a minefield ².

Developing effective detection, classification, and identification technologies for these hazards requires access to large amounts of data. Sonar, in its various forms, has been the primary sensing technology used in mine countermeasures (MCM) for decades, and sonar data is therefore essential for training and validating the machine learning and signal processing algorithms that underpin modern MCM systems

However, collecting real sonar data (whether in operational environments, controlled field trials, or laboratory settings) is costly, time-consuming, and in many cases dangerous. Synthetic data generation is therefore a valuable alternative, though not a trivial one: acoustic signals interact in complex ways with the seafloor, the water column, and the objects present in the scene, making realistic simulation a challenging task. Several sonar simulation tools exist, but each presents limitations that hinder their broad adoption for physics-based target modeling: some are proprietary or restricted to institutional use^{3,4}, while others leverage modern game engines to rapidly generate synthetic imagery for machine learning, often sacrificing rigorous elastic scattering physics for rendering speed⁵. Furthermore, while classical propagation models provide highly robust mathematical frameworks for sound transmission (e.g., Bellhop, Kraken)⁶, they lack the end-to-end architecture required to simulate complex target echoes.

The openSTB simulator⁷ addresses these challenges through two key characteristics: it is fully open-source, and it is built around a plugin architecture that allows researchers to adapt it to their specific needs. This flexibility means that the simulation pipeline can be extended by the community; while the core framework natively generates raw time-domain signals and multi-ping echograms, users can develop and integrate custom modules to support advanced processing chains, such as beamforming or full synthetic aperture sonar (SAS) imagery generation.

At the time of writing, the openSTB simulator supported two types of targets: ideal point scatterers, which model a target as a single omnidirectional reflector with no frequency or aspect dependence, adequate for validating processing chains or approximating large scenes composed of many reflectors, and two-dimensional arrangements of point scatterers, such as rectangles and triangles, which can approximate the acoustic response of simple seafloor features. However, no physics-based acoustic models were included for canonical targets such as spheres and cylinders. To the best of the authors' knowledge, no publicly available sonar simulator currently provides physics-based backscattering models for such targets.

Echoes generated by physics-based targets are particularly relevant in low-frequency active sonar applications, where the resonance and elastic vibration characteristics of the insonified object can be exploited to implement detection, classification, and identification algorithms^{8,9}. Unlike high-resolution imaging sonars, which rely on producing sharp acoustic images of the scene, low-frequency sonar can still yield discriminative echo signatures even when targets are partially or fully buried, or concealed beneath vegetation, rocks, or debris. In such conditions, image-based approaches typically fail or produce ambiguous results.

This paper presents the implementation of physics-based acoustic scattering models within the openSTB simulator pipeline. The model fully integrated and validated corresponds to a rigid sphere, whose backscattering response has been validated against classical analytical results¹⁰. Rigid acoustic scattering is governed by the geometry of the object rather than by its material composition or internal structure⁸, which makes the rigid sphere a physically motivated proxy for natural clutter such as rocks. Discriminating between natural objects and man-made targets is one of the primary tasks in mine countermeasure operations, making this type of model a relevant building block for synthetic sonar data generation. Additional models are available in the repository as validated standalone implementations and are currently being integrated into the simulator pipeline.

3 PLUGIN DESIGN AND IMPLEMENTATION

The implementation presented in this paper is available in the `scattering_models` branch of a fork of the openSTB simulator repository¹¹. The fork uses exclusively the *scripts* interface of the simulator, rather than the *command-line interface* (CLI) also provided by the framework. This choice was deliberate: the *scripts* interface exposes the full simulation configuration as a Python dictionary, allowing direct programmatic control over every parameter without the indirection of intermediate configuration files. To minimise interference with the upstream repository and to simplify the

incorporation of future updates from the original authors, all new files are contained within a dedicated `Experiments/` directory at the root of the repository, leaving the original source tree intact.

The contributions to the repository are organised into two groups. The first is a single file added to the simulator's source tree: `src/openstb/simulator/distortion/rigid_sphere.py`, which implements the `RigidSphereFormFunction` class. This class inherits from the framework's `Distortion` interface and applies the rigid sphere acoustic scattering response in the frequency domain, computing the form function from a modal series expansion based on spherical Bessel functions. The second group consists of the scripts in `Experiments/simple_points_study/`, which configure and drive the full simulation pipeline. The central file is `shared_params.py`, a single configuration dictionary that defines all simulation parameters — environmental conditions, sphere geometry, signal type, and diagnostic options — providing a single point of control for reproducibility. The function `signal_factory.py` reads these parameters and instantiates the appropriate signal object, either a linear frequency-modulated chirp or a sinusoidal burst. The script `simulate.py` assembles the complete simulation configuration, loading the `RigidSphereFormFunction` plugin as a distortion stage, and runs the simulation via the openSTB framework. Finally, `debug_plots.py` provides diagnostic visualisations of the incident signal, the form function extracted from the plugin, and the resulting echo.

The integration of the rigid sphere model into the openSTB pipeline exploits the framework's `Distortion` interface, through which plugins receive and return signals in the frequency domain. At each ping, the framework calls the `apply` method of `RigidSphereFormFunction`, passing the array of physical frequencies `f`, the complex signal spectrum `S` with shape $(N_{\text{receivers}} \times N_{\text{frequencies}} \times N_{\text{targets}})$, and a travel time result object that encapsulates the geometric configuration of the scene. From this object, the plugin retrieves the incident and scattering direction vectors for each target, and computes the scattering angle θ as the angle between them. The dimensionless frequency parameter ka is then obtained from the physical frequency array, the sphere radius, and the sound speed queried directly from the environment plugin. With these quantities, the form function is evaluated through a modal series expansion and returned as a complex array of the same shape as `S`. The output of the plugin is simply the element-wise product of the input spectrum and the form function, so that the framework can proceed with the inverse Fourier transform and produce the time-domain echo. This design means that the scattering physics are applied in the same domain in which the form function is naturally defined, without requiring any explicit time-domain convolution.

4 VALIDATION OF THE IMPLEMENTED MODELS

The validation strategy followed for all implemented models consisted of reproducing, as closely as possible, the results reported in the corresponding reference publications. Since the reference publications did not provide the numerical results themselves, and did not always provide the full set of input data, parameters, and implementation details required to reproduce them exactly, direct point-by-point comparison was not possible. Validation was therefore performed by assessing qualitative agreement between the outputs of the standalone scripts and the figures presented in the original papers. Once satisfactory agreement was achieved, the simulation parameters were varied systematically to observe the model behaviour under conditions beyond those reported in the references, providing additional confidence in the physical consistency of each implementation.

4.1 Overview of implemented models

The rigid sphere model¹⁰ was selected for first integration into the openSTB pipeline, as it represents one of the simplest cases and its architecture serves as the template for the remaining models. The other implemented models are currently available as standalone scripts in the repository and correspond to the following cases:

- **Elastic solid sphere in free field:** model based on Veksler ¹² and validated against Hickling¹³.
- **Vacuum-filled elastic spherical shell in free field:** model and validation based on Kargl and Marston ¹⁴.
- **Fluid-filled elastic spherical shell in free field:** model based on Gaunaurd and Werby ¹⁵, form function validated against Ayres et al. ¹⁶, and echo validated against Tesei et al. ⁹.
- **Rigid cylinder (infinite, normal insonification) in free field:** model and validation of individual modes based on Gaunaurd and Brill ¹⁷.
- **Elastic solid cylinder (infinite, normal insonification) in free field:** model based on Brill and Gaunaurd ¹⁸ and Gaunaurd and Brill ¹⁷, validated against the form function and individual modes reported by Brill and Gaunaurd ¹⁸ and by Veksler ¹².

4.2 Rigid sphere in free field

The rigid sphere backscattering model implemented in this work is based on the analytical formulation presented by Rudgers ¹⁰.

Rudgers' work addresses the problem of computing the scattered pressure field produced when a plane-wave pulse is incident upon an ideally rigid sphere immersed in a fluid. The goal of the paper is to demonstrate, through numerical computation, that the classical scattering theory yields results that can be interpreted in terms of the creeping-wave formulation of scattering theory. To this end, the scattered field is expressed analytically using a modal series expansion obtained by solving the boundary-value problem with Neumann boundary conditions at the sphere's surface, using separation of variables. The time-domain scattered pulse is then synthesized via a Fourier integral over the dimensionless frequency parameter ka , evaluated numerically using the trapezoidal rule. The incident pulseform considered is a truncated sinusoid. The results show that the scattered pulse can be understood as the superposition of two types of pulse components: a specularly reflected component, which resembles the incident pulse and arrives first, and a set of creeping-wave-type components, generated at the shadow boundary and travelling around the sphere's periphery before reaching the observation point.

To validate the implementation, three elements from the reference paper were selected for reproduction, in the following order: the incident signal (in both the time domain and the frequency domain, expressed in terms of the dimensionless parameter ka), the form function, and the resulting echo. This sequential approach allows the model to be verified progressively: confirming first that the input signal is correctly characterised, then that the frequency-domain response of the model agrees with the analytical result, and finally that the synthesized echo is consistent with the reference.

The incident signal is configured via `shared_params.py`. The acoustic medium is characterised by a sound speed in water of 1480 m/s, a salinity of 14.5 ppt, and a temperature of 11.2 °C. The sphere radius is set to 0.25 m and the target dimensionless frequency k_0a to 15.0. It should be noted that Rudgers does not specify the sphere radius or the sound speed used in his computations; the values adopted here are therefore chosen to be physically representative while yielding the desired k_0a . The resulting centre frequency is approximately 14.1 kHz, computed as $f_0 = k_0a \cdot c / (2\pi a)$. The incident signal type is selected by setting the `mode` parameter in `shared_params.py` to `sine`; the number of cycles is also defined there and set to 2, yielding a pulse duration of approximately 0.142 ms, computed as $T = n_cycles / f_0 = 2 / 14132.4 \approx 1.42 \times 10^{-4}$ s.

Several components of the simulator pipeline are mandatory and cannot be disabled: the trajectory, ping times, environment, travel time model, and target definition. To accommodate these requirements while keeping the scenario as simple as possible, a linear trajectory from [0, 0, 0] to [10, 0, 0] m at 1.5 m/s was defined in `simulate.py`, with pings transmitted at 0.1 s intervals. A single point target was placed at position (5, 40, 10) m with a reflectivity of 1.0, so as not to alter the scattering amplitude imposed by the rigid sphere model. The transmitter and the single receiver are positioned at [0, 1.2, 0.3] m and [0, 1.2, 0] m respectively, expressed in the AUV body frame; their

proximity approximates a monostatic configuration. The `random_point_rectangle` target type available in the simulator was deliberately left inactive, as it would introduce a simulated seabed, which is inconsistent with the free-field assumption of the reference model. Geometric spreading, Ainslie-McColm attenuation, and the transducer beam directivity pattern were all disabled in `simulate.py`, in order to isolate the rigid sphere scattering response under the idealized conditions of the analytical formulation.

It is also worth noting that Rudgers presents his results in terms of the dimensionless parameter ka , where k is the acoustic wavenumber and a is the sphere radius. This normalisation makes the results independent of the specific values of frequency and radius, and depends only on their combination. The present implementation operates in physical units, computing ka internally within the plugin from the physical frequency array and the sphere radius.

Figure 1 shows the incident signal obtained from the simulator in the time domain (a) and its spectrum expressed in terms of ka (b), both in excellent agreement with the results reported by Rudgers¹⁰. These figures are generated by `debug_plots.py`, after having first executed `simulate.py`.

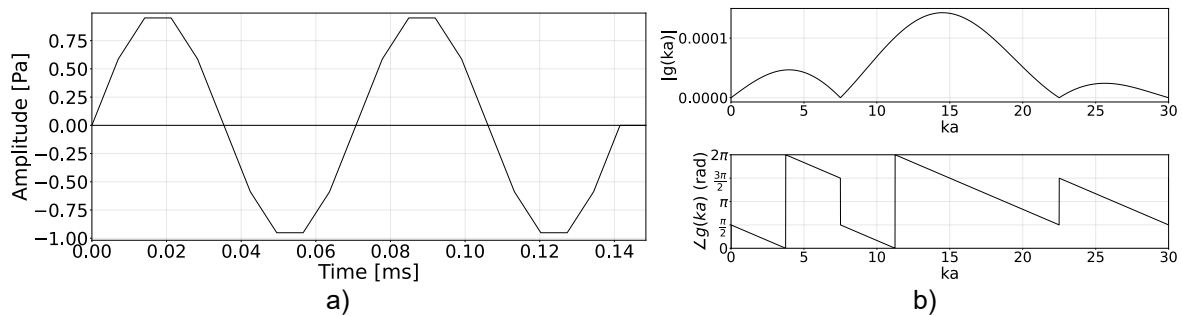


Figure 1: Incident signal obtained from `debug_plots.py`: (a) time-domain representation of the two-cycle truncated sinusoidal pulse; (b) magnitude (top) and phase (bottom) of the complex spectrum $g(ka)$, expressed in terms of the dimensionless frequency parameter ka .

The rigid sphere form function is implemented in the `RigidSphereFormFunction` class, defined in `src/openstb/simulator/distortion/rigid_sphere.py`. The expression evaluated is the far-field backscattering form function given by Rudgers¹⁰ (Eq. 11):

$$f(ka) = -\frac{2}{ka} \sum_{n=0}^{\infty} (2n+1) \sin(\eta_n) e^{i\eta_n} P_n(\cos \theta)$$

In the implementation, the series is truncated at $N = 80$, n is the modal index, P_n is the Legendre polynomial of order n , θ is the scattering angle, and η_n is the phase shift of the n -th partial wave, obtained from the derivatives of the spherical Bessel functions of the first and second kind:

$$\eta_n = \arctan\left(\frac{j'_n(ka)}{-y'_n(ka)}\right)$$

Compared to the standalone implementation, the plugin version introduces several adaptations required by the simulator pipeline. The sound speed c , needed to compute ka from the physical frequency array, is queried directly from the environment plugin at each ping, rather than being hardcoded. The scattering angle θ is computed from the incident and scattering direction vectors provided by the travel time result object, making the implementation valid for arbitrary geometries and not restricted to monostatic backscattering. To ensure numerical robustness, ka values below a threshold $ka_{eps} = 10^{-8}$ are replaced by that threshold before evaluating the modal series, and any residual NaN or Inf values are set to zero. The phase shift η_n is computed using `arctan2(j'_n, -y'_n)` rather than a direct division, which avoids instabilities when the denominator approaches zero.

Figure 2 (a) shows the magnitude and phase of $f(ka)$ extracted from the plugin during simulation, in excellent agreement with Fig. 2 of Rudgers¹⁰.

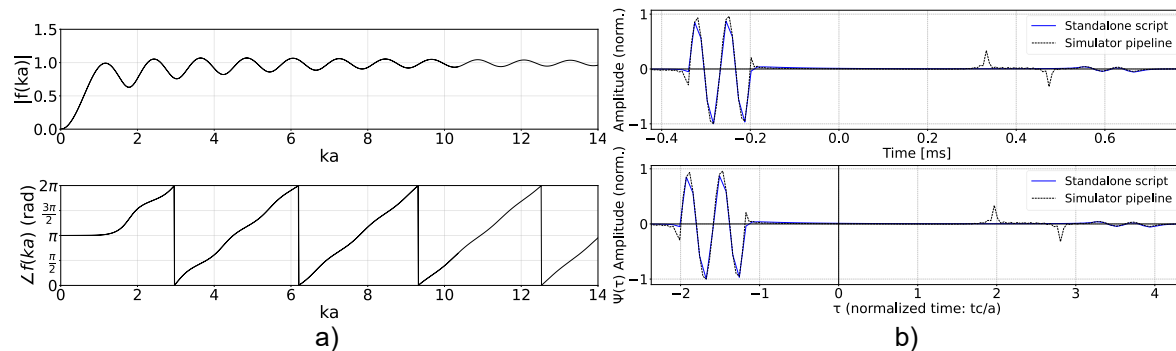


Figure 2: Results obtained from the rigid sphere simulation: (a) form function $f(ka)$ extracted from the `RigidSphereFormFunction` plugin, showing magnitude $|f(ka)|$ (top) and phase $\angle f(ka)$ (bottom), in excellent agreement with Fig. 2 of Rudgers¹⁰; (b) scattered echo in physical units (top) and normalised time units $\tau = tc/a$ (bottom), obtained from the standalone script `RigidSphereEcho.py`, comparing the standalone output (solid blue) and the simulator pipeline output (dashed black).

Figure 2(b) shows the scattered echo obtained from the standalone script `RigidSphereEcho.py`, located in `Experiments/scattering_models/rigid_sphere/free_field/`. Although the simulator fork also produces an echo, the standalone script is used here to enable a direct comparison between both outputs. The echo computation performed by the fork proceeds as follows: at each ping, the complex spectrum of the incident signal $S(f)$ is multiplied element-wise by the complex form function $f(ka)$, evaluated at the same physical frequencies, as implemented in the `apply` method of `RigidSphereFormFunction`. The result preserves both the magnitude and phase of the scattering response. The framework then applies an IFFT to recover the time-domain echo. The simulation uses a sample rate of $10 \times f_0 \approx 141$ kHz, with the number of frequency bins determined internally by the framework based on the ping window duration.

The echo is shown both in physical time units (top) and in the normalised time parameter $\tau = tc/a$ used by Rudgers (bottom), allowing direct comparison with Fig. 6 of Rudgers¹⁰. Both curves are produced by the standalone script `RigidSphereEcho.py`, but with different incident signal spectra as input: the solid blue curve uses the spectrum constructed internally by the standalone script from a real truncated sinusoid with a fixed FFT size, scaled by dt and a factor of 2, retaining only positive frequencies; the dashed black curve uses the complex spectrum $S(f)$ constructed internally by the simulator framework, dumped from `RigidSphereFormFunction` and read by the standalone script. The two versions are in close agreement for the main echo components, but the dashed black curve exhibits additional oscillations. Investigation confirms that these discrepancies do not originate from the rigid sphere form function itself — when the form functions computed by both implementations are overlaid, they are identical. The difference arises entirely from how each environment constructs $S(f)$: the simulator uses an analytic baseband signal and a dynamic FFT grid whose size depends on the ping window duration, extended by a guard band to avoid time-domain aliasing, whereas the standalone uses a simpler real-valued representation with a fixed grid. This confirms that the origin of the additional oscillations lies in the spectral construction pipeline and not in the scattering physics.

5 POINT SCATTERER VS PHYSICS-BASED TARGET

Having reproduced the reference results of Rudgers¹⁰ with the simulator, the scenario is now extended to use a linear frequency-modulated (LFM) chirp signal, which is more representative of the waveforms employed in active sonar systems for mine countermeasure applications. The signal is

selected by setting the `mode` parameter in `shared_params.py` to `lfm`; its characteristics are defined in `signal_factory.py` and consist of a frequency sweep from 0.5 kHz to 10 kHz, a duration of 20 ms, and a Hann window (other window types are available) applied to reduce spectral sidelobes. The simulation sampling rate defaults to 30 kHz in LFM mode. For a sphere of radius $a = 0.25$ m and a sound speed of 1480 m/s, this frequency sweep corresponds to a dimensionless frequency range of $ka \in [0.53, 10.6]$, spanning from the Rayleigh scattering regime through the resonance region and almost into the geometric regime —representing a range where the rigid sphere form function exhibits its most significant variation. Figure 3 shows the chirp signal in the time domain and its spectrum.

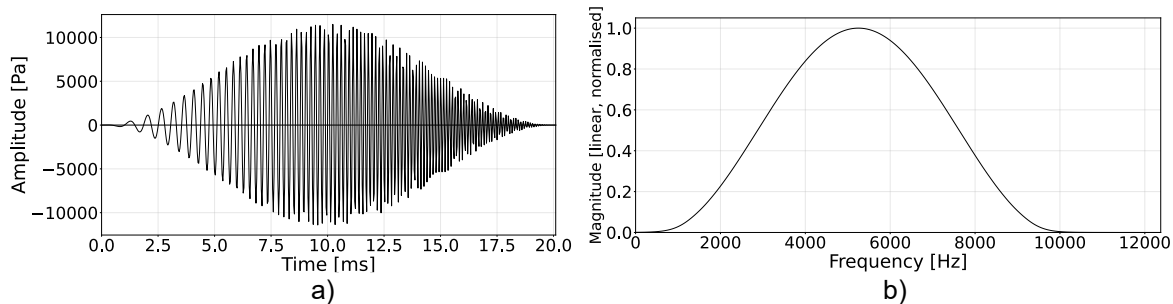


Figure 3: LFM chirp signal: (a) time-domain representation (passband view); the source level is set to 190 dB re 1 μ Pa RMS, corresponding to an RMS pressure of approximately 3162 Pa; a Hann window is applied and the signal is RMS-normalised over the full sampled record, so the time-domain peak reaches the order of 10000 Pa; (b) normalised magnitude spectrum, showing the signal energy concentrated between 0.5 kHz and 10 kHz.

The geometry, trajectory, and acoustic environment from Section 4.2 are kept unchanged: the AUV follows the same linear trajectory at 1.5 m/s with the same ping interval, the target is modelled as a single point scatterer with unit reflectivity (no extended or rectangular scatterer is used), and the water properties —sound speed, temperature, and salinity— remain as defined in `shared_params.py` and are handled by the invariant environment plugin. The `stop-and-hop` travel time approximation used in Section 4.2 is replaced here by the `iterative` model, which accounts for the actual motion of the vehicle during each ping by iteratively solving for the reception time until convergence, yielding a more physically accurate result for a moving platform.

In this scenario, the geometric spreading, Ainslie-McColm attenuation, transducer directivity, and Doppler distortions are all activated. The geometric spreading plugin models the amplitude decay of the acoustic wave as it propagates, applying a $1/r$ loss to both the transmitted and received paths, corresponding to spherical spreading. The Ainslie-McColm attenuation plugin models frequency-dependent absorption in seawater using the simplified formula of Ainslie and McColm¹⁹, which accounts for the chemical relaxation of boric acid (dominant below 1 kHz), the relaxation of magnesium sulphate (dominant from 10 kHz to 100 kHz), and viscous drag (dominant above 200 kHz); given the short ranges and moderate frequencies involved, its effect is expected to be small.

The transducer directivity is modelled using the `RectangularBeampattern` plugin, which computes the far-field directivity of an ideal rectangular aperture as a 2D sinc function of the angle from the transducer boresight, parameterized by the aperture width (0.6 m) and height (0.8 m), with the transducers oriented at a yaw of $+90^\circ$ and a tilt of $+15^\circ$ relative to the AUV body frame. The frequency parameter is set to `all`, meaning the directivity gain is evaluated independently at each frequency bin of the signal spectrum, rather than using a single effective gain computed at the centre frequency; this produces a more physically accurate result, as the beamwidth of a finite aperture narrows with increasing frequency. Finally, the Doppler distortion plugin models the frequency shift induced by the motion of the platform, computing a scale factor $\eta = (1 + v_{rx}/c)/(1 - v_{tx}/c)$, where v_{tx} and v_{rx} are the projector and hydrophone velocities in the direction of the target, and resampling the spectrum accordingly.

Figure 4 presents the simulation results produced by `plot_results.py` with all distortions active, including the rigid sphere form function. Figure 4(a) shows the echo level as a function of time for ping 31, which corresponds approximately to the closest point of approach. The echo arrives at approximately 53 ms, consistent with the target range and the platform position at that ping. Figure 4(b) shows the echogram across all 62 pings. The target trace follows a hyperbolic curve which appears nearly straight due to the large target range relative to the platform displacement. The echogram also shows a darker horizontal band near ping 33. This is not a weakening of the echo at that ping but rather a visualisation artefact: the echogram is computed as the instantaneous dB level across the entire time trace, and numerical zeros and phase interference outside the main echo window pull the row average down in logarithmic scale. Tests confirmed this: when the rigid sphere distortion and Doppler were deactivated, the dark band shifted to ping 30, while the echo peak remained strong near ping 33. A future improvement would be to restrict the dB computation to a window around the echo, or to use an energy-based metric per ping. When the rigid sphere distortion is deactivated and a point scatterer model is used instead, the echogram and single-ping trace show only minor differences at this scale; the effect of the form function is more evident when examining the `debug_plots.py` output, particularly the echo waveform in linear scale and its frequency spectrum, as discussed in the previous section.

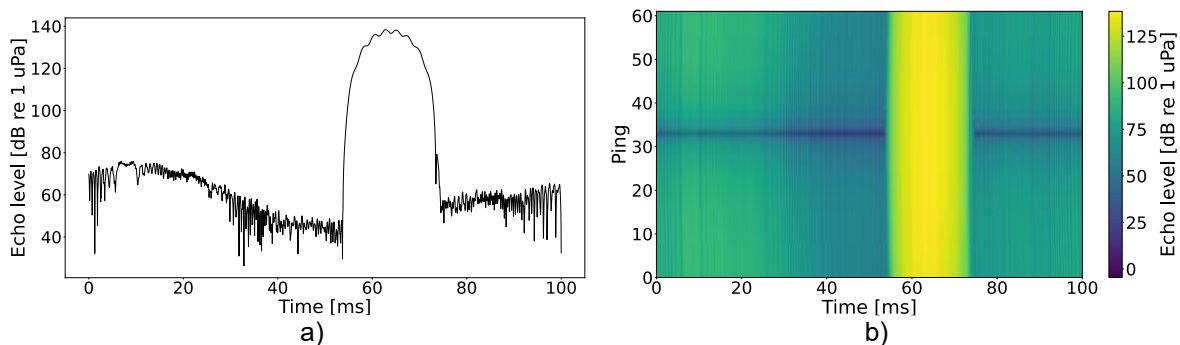


Figure 4. Simulation results with all distortions active (geometric spreading, Ainslie-McColm absorption, Doppler shift, rectangular beampattern, and rigid sphere form function). (a) Echo level as a function of time for ping 31. (b) Echogram over 62 pings; the target trace follows a hyperbolic curve that appears nearly straight due to the large target range.

Figure 5 shows the output of `debug_plots.py` for ping 31. Figure 5(a) presents the received echo in the time domain in linear units (Pa), and Figure 5(b) shows its normalised magnitude spectrum. Both panels confirm that the echo is a distorted version of the transmitted chirp: the time-domain waveform retains the swept-frequency structure of the incident signal (Figure 3a) but with an amplitude envelope shaped by the combined effect of geometric spreading, absorption, beampattern, and especially, rigid sphere scattering. The echo arrival time is consistent with the expected two-way travel time given the platform and target positions. The spectral effect of the rigid sphere form function is clearly visible in Figure 5(b): unlike the smooth bell-shaped spectrum of the incident signal (Figure 3b), the echo spectrum exhibits an irregular shape with lobes and a pronounced roll-off, reflecting the frequency-dependent modulation imposed by $f(ka)$ over the range $ka \in [0.53, 10.6]$. This spectral coloration is the primary physical signature of the rigid sphere model, and explains why the difference between a physics-based scatterer and a point scatterer is subtle at the echogram level but clearly apparent in the spectral analysis of individual echoes.

6 CONCLUSIONS

This paper presented the implementation and validation of a rigid sphere acoustic scattering model as a distortion plugin for the openSTB sonar simulator. The far-field form function derived by Rudgers¹⁰ was reproduced successfully, confirming that the modal series expansion and the associated numerical procedures are correctly implemented. When the simulator is extended to use

an LFM chirp signal with a realistic set of distortions —geometric spreading, frequency-dependent absorption, transducer directivity, and Doppler shift— the received echo departs from the incident waveform in a manner that is consistent with the physics of rigid sphere scattering: the form function imprints a characteristic spectral coloration on the echo, which is clearly visible in the frequency domain even when its effect on the echogram is subtle.

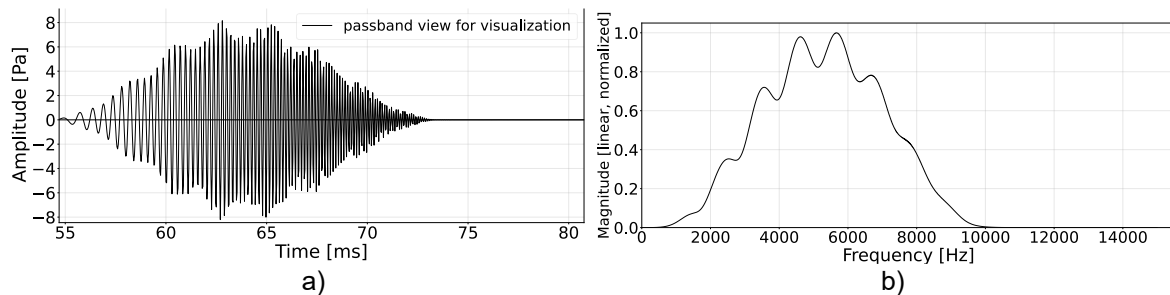


Figure 5. Echo signal for ping 31 produced by `debug_plots.py` with all distortions active. (a) Time-domain waveform in linear units; (b) normalised magnitude spectrum, showing the spectral coloration imposed by the rigid sphere form function over the frequency range of the LFM chirp.

To the authors' knowledge, openSTB may represent the first open sonar simulator to incorporate physics-based elastic scattering models of this kind. The plugin architecture of the framework makes it straightforward to activate, deactivate, and combine distortion models, enabling a wide range of controlled experiments without modifying the core simulation engine. Additional scattering models for other target geometries are currently available as standalone scripts and are being progressively integrated into the simulator.

At the time of writing, the scattering model operates under free-field conditions, which limits its applicability to scenarios without a seabed or surface boundary. As future work, the authors are developing an implementation based on Kargl, Williams and Thorsos²⁰ for simulating the scattering of spheres above a flat surface, which would extend the simulator to more operationally realistic environments. Further planned extensions include noise plugins, matched filtering, beamforming, synthetic aperture sonar processing, and support for multiple receivers. Finally, the current implementation targets the script-based execution mode of openSTB; integration with the TOML-based command-line interface is also envisioned, which could open additional workflows for users who prefer a configuration-file-driven approach.

More broadly, while the literature on acoustic scattering from canonical shapes is extensive, it remains difficult to navigate. No concise reference exists that consolidates the key resulting equations in a form accessible to newcomers; existing works such as Veksler¹² are comprehensive but lengthy. Beyond this, different publications adopt inconsistent conventions, variable definitions, and reference frames, and subtle errors in the form function equations —a missing derivative, an incorrect sign— persist across papers and even textbooks, occasionally addressed by later publications but never fully resolved. These inconsistencies caused significant difficulties during implementation and consumed considerable time before correct replication of the reference results was achieved. A consolidated and carefully verified reference for canonical scattering models would therefore be a valuable contribution to the community.

ACKNOWLEDGEMENTS

This work was conducted under the grant C-8670 funded by the Walloon Region in Belgium.

7 REFERENCES

1. Beldowski J, Klusek Z, Szubska M, et al. Chemical Munitions Search & Assessment—An evaluation of the dumped munitions problem in the Baltic Sea. *Deep Sea Research Part II: Topical Studies in Oceanography*. 2016;128:85-95. doi:10.1016/j.dsr2.2015.01.017
2. Khaledi S, Mann H, Perkovich J, Zayed S. Design of an underwater mine detection system. In: *2014 Systems and Information Engineering Design Symposium (SIEDS)*. 2014:78-83. doi:10.1109/SIEDS.2014.6829901
3. Goddard RP. *The Sonar Simulation Toolset, Release 4.6: Science, Mathematics, and Algorithms*. Applied Physics Laboratory, University of Washington; 2008. <https://apps.dtic.mil/sti/html/tr/ADA488253/>
4. Woods D. An overview of the MASTODON sonar simulator. *J Acoust Soc Am*. 2023;154(4_supplement):A211. doi:10.1121/10.0023310
5. Almanza-Medina JE, Henson BT, Zakharov YV. Imaging sonar simulator for assessment of image registration techniques. In: *OCEANS 2019 MTS/IEEE SEATTLE*. 2019:1-7. doi:10.23919/OCEANS40490.2019.8962864
6. Jensen FB, Kuperman WA, Porter MB, Schmidt H. *Computational Ocean Acoustics*. Springer; 2011. doi:10.1007/978-1-4419-8678-8
7. Bonnett B, Schmaljohann H, Apte S, Fickenschner T. Current state of the openSTB sonar simulator. In: *Proceedings of the Institute of Acoustics*. Institute of Acoustics; 2025. Accessed April 13, 2026. https://www.uaconferences.org/docs/2025_proceedings/367_UACE2025_Proceedings.pdf
8. Jia H, Li X, Meng X. Rigid and elastic acoustic scattering signal separation for underwater target. *J Acoust Soc Am*. 2017;142(2):653-665. doi:10.1121/1.4996127
9. Tesei A, Fawcett JA, Lim R. Physics-based detection of man-made elastic objects buried in high-density-clutter areas of saturated sediments. *Applied Acoustics*. 2008;69(5):422-437. doi:10.1016/j.apacoust.2007.04.002
10. Rudgers AJ. Acoustic Pulses Scattered by a Rigid Sphere Immersed in a Fluid. *The Journal of the Acoustical Society of America*. 1969;45(4):900-910. doi:10.1121/1.1911567
11. Hurtado Erasso CA. [camihurs/simulator](https://github.com/camihurs/simulator). Published online March 6, 2026. Accessed April 22, 2026. <https://github.com/camihurs/simulator>
12. Veksler ND. *Resonance Acoustic Spectroscopy*. Vol 11. (Überall H, ed.). Springer; 1993. doi:10.1007/978-3-642-84795-0
13. Hickling R. Analysis of Echoes from a Solid Elastic Sphere in Water. *J Acoust Soc Am*. 1962;34(10):1582-1592. doi:10.1121/1.1909055
14. Kargl SG, Marston PL. Ray synthesis of the form function for backscattering from an elastic spherical shell: Leaky Lamb waves and longitudinal resonances. *J Acoust Soc Am*. 1991;89(6):2545-2558. doi:10.1121/1.400694
15. Gaunard GC, Werby MF. Lamb and creeping waves around submerged spherical shells resonantly excited by sound scattering. *J Acoust Soc Am*. 1987;82(6):2021-2033. doi:10.1121/1.395646
16. Virginia M, Ayres, Ayres VM, G. C. Gaunard, et al. The effects of lamb waves on the sonar cross-sections of elastic spherical shells. *International Journal of Solids and Structures*. 1987;23(7):937-946. doi:10.1016/0020-7683(87)90088-6
17. Gaunard GC, Brill D. Acoustic spectrogram and complex-frequency poles of a resonantly excited elastic tube. *J Acoust Soc Am*. 1984;75(6):1680-1693. doi:10.1121/1.390907
18. Brill D, Gaunard GC. Acoustic resonance scattering by a penetrable cylinder. *The Journal of the Acoustical Society of America*. 1983;73(5):1448-1455. doi:10.1121/1.389432
19. Ainslie MA, McColm JG. A simplified formula for viscous and chemical absorption in sea water. *J Acoust Soc Am*. 1998;103(3):1671-1672. doi:10.1121/1.421258
20. Kargl SG, Williams KL, Thorsos EI. Synthetic Aperture Sonar Imaging of Simple Finite Targets. *IEEE Journal of Oceanic Engineering*. 2012;37(3):516-532. doi:10.1109/JOE.2012.2200815